

CHUNKING STRATEGIES

A Practical Guide to Text Segmentation

From fundamental splitting methods to advanced, adaptive, and AI-driven techniques.

15 Strategies | Technical Concepts | Practical Guidelines

Nishanth Gadey

WHAT IS CHUNKING?

Turning Documents into Meaningful Units

Chunking is the process of dividing large documents into smaller, manageable text segments based on size, structure, or semantic meaning.

Instead of treating an entire document as one unit, chunking creates smaller sections that can be processed independently while preserving their original meaning.

Why it matters

- Makes large documents easier to process.
- Preserves meaningful units of information.
- Controls text length for downstream models.
- Enables consistent indexing, storage, and source tracking.

Core objective: Create chunks that are small enough to process efficiently but large enough to preserve meaning.

THREE CORE PRINCIPLES

What Makes a Good Chunk?

1. Semantic Coherence

Keep related concepts together. Avoid combining unrelated topics or separating closely connected statements unnecessarily.

---

2. Context Preservation

Retain the headings, definitions, conditions, and surrounding information needed to understand a chunk independently.

---

3. Computational Efficiency

Balance chunk size, processing time, storage, token limits, and the cost of generating and maintaining chunks.

The trade-off: Smaller chunks provide finer granularity; larger chunks preserve more context. Neither is universally better.

UNDERSTANDING CHUNK SIZE

How Much Text Should a Chunk Contain?

Chunk size determines how much content is grouped into each segment. It may be measured in characters, words, or tokens.

Small Chunks

- More granular information.
- Less surrounding context.
- More chunks to process and maintain.

Large Chunks

- More complete explanations.
- Greater risk of combining unrelated topics.
- Higher processing and context requirements.

Practical starting point
Experiment with 256, 512, and 1,024 tokens where supported by the model and application. These are experimental starting values, not universal defaults.

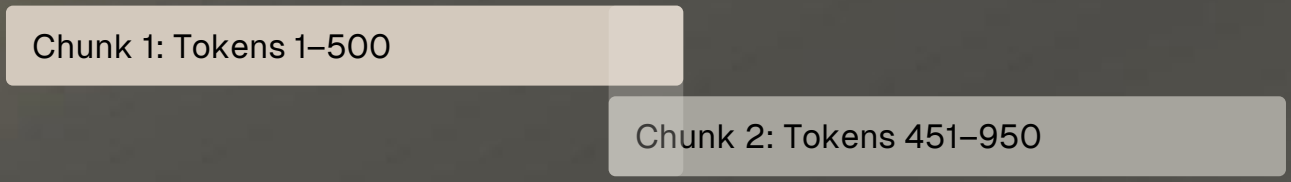
Rule: Measure token length using the appropriate tokenizer and respect the model's actual input limit.

CHUNK OVERLAP

Preserving Context Across Boundaries

Overlap repeats part of one chunk in the next to reduce information loss near splitting boundaries.

Example



Overlap: 50 tokens

Benefits

- Preserves boundary context.
- Reduces the chance of separating connected statements without shared context.
- Can help when information spans adjacent chunks.

Trade-offs

- Duplicates content.
- Increases processing and storage requirements.
- May introduce redundant text into downstream processing.

Best practice: Start with zero overlap as a baseline, then test modest overlap when boundary-related failures justify it.

FACTORS THAT DETERMINE YOUR STRATEGY

No Single Strategy Fits Every Document

Document structure: Plain text, Markdown, PDFs, code, tables, or mixed content.

Content complexity: Simple prose versus dense technical explanations.

Boundary sensitivity: Whether splitting a sentence, table, clause, or code block damages its meaning.

Length constraints: Maximum supported input tokens and downstream context requirements.

Update frequency: Whether documents change frequently and chunks must be regenerated.

Processing budget: Available compute, preprocessing time, and storage.

Consistency requirements: Whether reproducible, deterministic chunk boundaries are important.

Choose according to the content and constraints—not popularity alone.

STRATEGY 1

FIXED-SIZE CHUNKING

Divide by a Predetermined Length

One-line concept

Splits text into segments of a predefined size measured in characters, words, or tokens.

How it works

Choose a target size N and optionally an overlap O. Generate successive windows, advancing approximately N minus O units per chunk.

Best for: Plain text, simple logs, and uniform documents.

Pros

- Simple to implement.
- Predictable chunk sizes.
- Efficient for uniform text.

Cons

- May split sentences or ideas.
- Ignores natural boundaries.
- Can separate related information.

Related variant — Sliding Window: Uses a moving window with consistent overlap between consecutive chunks.

STRATEGY 2

SENTENCE-BASED CHUNKING

Preserve Complete Sentences

One-line concept

Groups complete sentences into chunks while respecting a target size whenever possible.

How it works

Use a sentence tokenizer to identify boundaries, then group consecutive sentences until the configured word or token budget is reached.

Best for: Articles, conversational text, general prose, and sentence-oriented content.

Pros

- Preserves complete thoughts.
- Respects natural sentence boundaries.
- Works well for ordinary prose.

Cons

- Produces variable chunk sizes.
- Sentence lengths can differ significantly.
- Sentence boundaries do not always match topic boundaries.

STRATEGY 3

PARAGRAPH-BASED CHUNKING

Use Natural Paragraph Boundaries

One-line concept

Treats each paragraph as a natural unit of meaning.

How it works

Detect paragraph separators and create chunks from individual paragraphs. Combine short paragraphs or split oversized ones when necessary.

Best for: Articles, blogs, emails, books, and well-formatted documentation.

Pros

- Preserves related sentences.
- Follows the author's organization.
- Requires relatively little preprocessing.

Cons

- Paragraph sizes vary.
- Long paragraphs may exceed model limits.
- Depends on clean document formatting.

STRATEGY 4

RECURSIVE CHUNKING

Split Large Structures Before Smaller Ones

One-line concept

Uses a hierarchy of separators, falling back to smaller boundaries when a section is too large.

How it works

A typical separator hierarchy is:

Headings → Paragraphs → Lines → Sentences → Words → Characters

Oversized segments are split recursively until they satisfy the configured size limit.

Best for: General-purpose text, scraped websites, mixed-format content, and technical documentation.

Pros

- Adapts to inconsistent documents.
- Preserves larger semantic units when possible.
- Supports configurable fallback rules.

Cons

- Requires separator tuning.
- Generic rules may not suit every content type.
- Oversized structural elements still need special handling.

STRATEGY 5

DOCUMENT-BASED CHUNKING

Preserve Document Hierarchy

One-line concept

Creates chunks using structural elements such as headings, sections, lists, and document boundaries.

How it works

Parse the document into a structural representation. Create chunks from appropriate sections or leaf elements and include relevant parent headings for context.

Best for: Reports, policies, Markdown, HTML, technical documentation, and structured PDFs.

Pros

- Preserves section hierarchy.
- Retains relationships between headings and content.
- Works well with structured documents.

Cons

- Requires parsing and preprocessing.
- Section sizes can vary significantly.
- Large sections still need fallback splitting.

STRATEGY 6

SEMANTIC CHUNKING

Split When Meaning Changes

One-line concept
Uses semantic differences between neighboring text segments to identify meaningful chunk boundaries.

How it works
Split text into sentences, generate sentence embeddings, and compare adjacent sentences or groups using cosine similarity or another semantic-distance measure. Create a boundary when the configured threshold indicates a sufficiently large semantic change.

Best for: Research papers, long technical documents, transcripts, and content with frequent topic changes.

Pros

- Adapts to topic transitions.
- Can produce stronger topical coherence.
- Reduces reliance on arbitrary boundaries.

Cons

- Requires additional embedding computations.
- Thresholds need tuning Slower processing.
- Chunk sizes can vary substantially. Require additional model inference

STRATEGY 7

: HIERARCHICAL PARENT–CHILD CHUNKING

Combine Fine Detail with Broader Context

One-line concept
Organizes large parent chunks and smaller child chunks into a hierarchy.

How it works
Divide a document into parent sections and smaller child segments. Maintain identifiers linking each child to its parent so the broader section can be retrieved when necessary.
Example: A 1,024-token parent section containing several 256-token children.

Best for: Large documentation sets, research papers, enterprise knowledge bases, and lengthy reports.

Pros

- Supports multiple levels of granularity.
- Preserves broader section context.
- Enables flexible context expansion.

Cons

- Requires relationship management.
- Updates can affect multiple linked chunks.
- Expanding parent context can increase processing cost.

STRATEGY 8

ADAPTIVE CHUNKING

Let Content Characteristics Guide the Split

One-line concept
Selects chunk sizes or chunking methods according to document characteristics.

How it works
Analyze structure, complexity, length, and content type. Use rules, classifiers, or evaluation-driven selection to choose suitable chunking configurations for different sections.

Best for: Mixed-content document collections, technical manuals, and large knowledge bases.

Pros

- Adapts to heterogeneous content.
- Can balance size and semantic coherence.
- Reduces reliance on one global configuration.

Cons

- Requires strategy-selection logic.
- More difficult to tune and debug.
- May increase preprocessing costs.

STRATEGY 9

CONTEXT-ENRICHED CHUNKING

Make Chunks Understandable on Their Own

One-line concept
Adds relevant contextual information to a chunk before it is represented or processed independently.

How it works
Attach document titles, section headings, parent-section summaries, dates, or other relevant metadata. Advanced implementations may generate a chunk-specific contextual description using an LLM.

Best for: Multi-section reports, research papers, compliance documents, and content with cross-section dependencies.

Pros

- Clarifies ambiguous references.
- Preserves important document context.
- Supports source traceability.

Cons

- Adds tokens and storage requirements.
- Can introduce redundant information.
- Generated descriptions require validation.

Important: Metadata enrichment and LLM-generated contextual descriptions are related but distinct implementations.

STRATEGY 10

MODALITY-SPECIFIC CHUNKING

Respect Tables, Images, Charts, and Other Structures

One-line concept

Processes different content types according to their structures while maintaining their relationships to the source document.

How it works

Use document parsers, OCR, table extraction, or vision-language models to identify content types. Preserve table headers, figure captions, page references, and links between text and visual elements.

Best for: Financial reports, scientific papers, scanned PDFs, charts, and multimodal documents.

Pros

- Preserves non-text structure.
- Maintains contextual and source references.
- Handles complex document layouts.

Cons

- Requires specialized processing.
- Increases pipeline complexity.
- Relationships can be lost if not explicitly maintained.

STRATEGY 11

LLM-DRIVEN CHUNKING

Let an LLM Identify Meaningful Boundaries

One-line concept
Uses a large language model to identify logical segments based on document meaning and structure.

How it works
Provide the document or manageable sections to an LLM. Request structured output containing chunk text, logical boundaries, or atomic propositions. Validate completeness, ordering, token limits, and source fidelity.

Best for: Complex contracts, regulatory documents, research papers, and content requiring specialized segmentation.

Pros

- Can identify context-aware boundaries.
- Adapts to complex content.
- Can preserve meaningful facts and relationships.

Cons

- Increases inference costs.
- Outputs may be inconsistent.
- Requires robust validation.

STRATEGY 12

AGENTIC ORCHESTRATION-DRIVEN CHUNKING

Coordinate the Entire Chunking Workflow

One-line concept

Uses an agentic workflow to select, execute, evaluate, and refine chunking strategies.

How it works

An orchestrator analyzes document characteristics, chooses suitable methods, invokes parsers or LLMs, validates the output, and optionally evaluates results before refining the configuration.

Best for: Heterogeneous enterprise collections and complex ingestion pipelines.

Pros

- Adapts workflows to different document types.
- Combines deterministic and AI-driven methods.
- Supports automated evaluation and refinement.

Cons

- Adds orchestration complexity.
- Can increase latency and cost.
- Requires monitoring, validation, and debugging.

Key distinction: LLM-driven chunking determines boundaries; agentic orchestration manages the broader decision-making workflow.

STRATEGY 13

LATE CHUNKING

Contextualize First, Pool Later

One-line concept

Generates contextualized token embeddings from a larger text span before pooling them into individual chunk vectors.

How it works

A compatible long-context embedding model processes the larger text span and produces token-level representations. These representations are then grouped and pooled according to chunk boundaries.

Best for: Long documents and content where a passage's meaning depends on surrounding text.

Pros

- Incorporates surrounding context into chunk representations.
- Can improve representation of cross-sentence references.
- Avoids independently encoding every chunk.

Cons

- Requires compatible model capabilities.
- Long-context processing can be expensive.
- Requires accurate token-to-chunk alignment.

STRATEGY 14

PROPOSITION-BASED CHUNKING

Break Complex Text into Atomic Facts

One-line concept
Decomposes paragraphs into small, self-contained factual statements.

How it works
Use an LLM or extraction pipeline to identify individual propositions. Preserve entities, conditions, qualifiers, and source references. Store the propositions individually or in small groups.

Best for: Fact-heavy knowledge bases, scientific documents, and precise question-answering applications.

Pros

- Enables fine-grained factual representation.
- Separates unrelated claims.
- Can preserve precise statements for targeted processing.

Cons

- Decomposition can lose qualifications.
- LLM processing increases costs.
- Produces more records and requires source mapping.

STRATEGY 15

CODE-AWARE CHUNKING

Split Code by Programming Structure

One-line concept

Uses programming-language structures rather than ordinary prose boundaries.

How it works

Parse code using an Abstract Syntax Tree (AST) or language-aware rules. Create chunks around functions, classes, methods, and logical blocks while retaining signatures, imports, file paths, and relevant symbol metadata.

Best for: Code repositories, developer assistants, code documentation, and programming-focused applications.

Pros

- Preserves code structure.
- Supports function- and class-level granularity.
- Retains useful navigation metadata.

Cons

- Requires language-aware parsing.
- Large functions may still need splitting.
- Cross-file dependencies require additional handling.

ADVANCED TECHNIQUES & EMERGING TRENDS

Beyond Conventional Text Splitting

Domain-specific chunking
Adapts boundaries to legal clauses, financial tables, medical records, or technical structures.

Multimodal chunking
Preserves relationships between text, images, charts, tables, and other content types.

Query-aware granularity
Uses different levels of detail depending on the information required.

Neural segmentation
Uses learned models to identify boundaries rather than relying exclusively on manual rules.

Hierarchical processing
Maintains relationships between documents, sections, paragraphs, and smaller passages.

Contextual enrichment
Adds source-derived context to make isolated chunks more meaningful.

Important: These techniques have different maturity levels and implementation requirements. Evaluate their value for the specific application instead of treating them as universally superior methods.

PRACTICAL STRATEGY SELECTION

Which Strategy Should You Choose?

Selection process

1. Identify document types.
2. Understand the content boundaries that matter.
3. Establish a simple baseline.
4. Test candidate configurations.
5. Diagnose failures.
6. Introduce targeted improvements.

Choose the simplest strategy that satisfies the requirements.

Content type	Starting strategy
Plain text and simple logs	Fixed-size or recursive
Articles and blogs	Paragraph-based
Markdown and HTML	Structure-aware
Long research papers	Structure-aware or recursive
Frequent topic changes	Semantic
Long sections requiring context	Parent–child
Mixed document types	Adaptive or hybrid
Legal and regulatory documents	Structure-aware or validated LLM-driven
Tables and scanned documents	Modality-specific
Code repositories	Code-aware
Fact-heavy content	Proposition-based
Context-dependent passages	Context-enriched or late chunking

CHUNKING STRATEGY COMPARISON

Understand the Trade-offs

Fixed-size chunking with sliding windows is a common variant of the fixed-size approach.

Strategy	Main advantage	Main limitation
Fixed-size	Simplicity	Arbitrary boundaries
Sentence-based	Complete sentences	Variable lengths
Paragraph-based	Natural organization	Inconsistent sizes
Recursive	Flexible fallback rules	Separator tuning
Document-based	Preserves hierarchy	Parsing complexity
Semantic	Topic-aware boundaries	Additional computation
Parent–child	Multiple granularities	Relationship management
Adaptive	Content-specific selection	Selection complexity
Context-enriched	Additional context	More tokens
Modality-specific	Preserves content structures	Specialized processing
LLM-driven	Meaning-aware segmentation	Inference cost
Agentic orchestration	Workflow automation	Operational complexity
Late chunking	Contextualized vectors	Model dependency
Proposition-based	Fine-grained facts	Decomposition risk
Code-aware	Programming structure	Language-specific parsing

BEST PRACTICES

Build a Reliable Chunking Pipeline

1. Start with a baseline
Use fixed-size, recursive, or structure-aware splitting before introducing advanced methods.

2. Respect natural boundaries
Preserve headings, paragraphs, complete statements, tables, and code blocks where possible.

3. Enforce token limits
Use the appropriate tokenizer and validate every chunk before processing.

4. Use overlap selectively
Measure its benefit rather than assuming more overlap is always better.

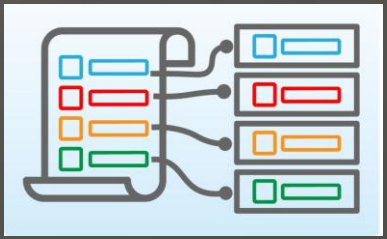
5. Preserve source metadata
Maintain document identifiers, versions, section paths, page numbers, and source offsets.

6. Handle special content explicitly
Use dedicated parsing for tables, code, scanned documents, and multimodal content.

7. Make processing reproducible
Version configurations and validate AI-generated segmentation.

8. Test real examples
Inspect boundary errors, missing information, excessive duplication, and oversized chunks.

END-TO-END CHUNKING WORKFLOW



From Raw Documents to Validated Chunks

STEP 1 — Ingest
Collect source documents and record identifiers and versions.

STEP 2 — Extract
Extract text, headings, tables, images, code, and source locations.

STEP 3 — Analyze
Identify document structure and content types.

STEP 4 — Select
Choose the chunking method and size constraints.

STEP 5 — Split
Generate chunks using natural boundaries and appropriate fallback rules.

STEP 6 — Enrich
Add relevant headings, metadata, or contextual descriptions when needed.

STEP 7 — Validate
Check token limits, content integrity, ordering, duplication, and source mappings.

STEP 8 — Store
Save chunks, metadata, relationships, and configuration versions.

STEP 9 — Evaluate
Compare chunk quality, processing costs, and downstream performance.

STEP 10 — Refine
Adjust the configuration based on observed failures.

EVALUATING CHUNKING QUALITY

Measure More Than Chunk Size

Structural quality

- Are logical sections preserved?
- Are tables and code blocks intact?
- Are important relationships maintained?

Integrity

- Are important source passages missing?
- Are there accidental duplicates?
- Can every chunk be traced to its source?

Size distribution

- Average and maximum token counts.
- Number of very small or oversized chunks.
- Distribution of chunk lengths.

Operational efficiency

- Processing time.
- Number of chunks produced.
- Storage and computational cost.

Downstream effectiveness
Test the chunks using a representative set of real questions and compare the quality of the resulting answers.

A visually clean chunk is not necessarily an effective chunk. Evaluate whether the segmentation preserves the information the application actually needs.

COMMON MISTAKES

What to Avoid

Using one chunk size for every document
Different content types may require different boundaries.

Splitting only by character count
Character counts do not reliably correspond to token counts.

Ignoring document structure
Headings, lists, tables, and code may carry important meaning.

Using excessive overlap
Repeated content can increase processing costs and redundancy.

Adopting advanced techniques without evaluation
Complexity alone does not guarantee better results.

Discarding source references
Without accurate source mappings, traceability and verification become difficult.

Ignoring updates
Changed documents can leave stale chunks or inconsistent relationships in the index.

Better approach
Establish a baseline, identify measurable problems, and improve the specific stage responsible.

KEY TAKEAWAYS

Principles to Remember

1. Chunking determines how information is segmented.
2. Semantic coherence and context preservation matter more than uniformity alone.
3. Token limits should guide size constraints.
4. Overlap is a trade-off, not a requirement.
5. Document structure should influence the splitting method.
6. Advanced strategies solve specific problems; they are not automatically better.
7. Contextual, hierarchical, and multimodal techniques require appropriate metadata and validation.
8. Evaluation should guide configuration choices.
9. Reproducibility and source traceability matter in production systems.
10. The best chunking strategy depends on the documents, requirements, and measured results.

The objective is not to create perfectly uniform chunks. It is to preserve the right information in the right units.

Better Chunking Starts with Better Decisions.

- Understand the document.
- Preserve meaningful boundaries.
- Choose the right level of granularity.
- Validate with real examples.
- Improve through measurement.

Chunking is not just splitting text. It is designing meaningful information units.

Nishanth Gadey